VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

STUDENT RESULT MANAGEMENT SYSTEM USING PYTHON

[A9513] – Python Programming

Submitted by

G.AKASH
Roll No: 25881A0568

E.MANIKANTA
Roll No: 25881A0595

Course Facilitator

Ms MADHAVI
Assistant Professor

Department of Computer Science and Engineering
Vardhaman College of Engineering, Hyderabad

Autonomous Institute, Affiliated to **JNTUH**, Approved by **AICTE**, Accredited by **NAAC** with **A++** Grade
Kacharam, Shamshabad, Hyderabad - 501 218, Telangana, India.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Course End Project titled **"Student Result Management System Using Python"** is carried out by G.AKASH, E.MANIKANTA with Roll Numbers 25881A0568, 25881A0595 towards A9513 **– Python Programming Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2025-26.

Signature of the Course Faculty
Ms MADHAVI
Assistant Professor, CSE

Signature of the HOD
Dr. Y.VIJAYALATA
Professor,Head of the Department & Dean (student Mentoring)
Department Computer Science and Engineering,Vardhaman College of Engineering,Hyderabad

Contents

1. Introduction **1**
2. Literature Review **1**
3. Objectives **2**
4. Methodology **3**
4.1 Architectural Diagram 3
4.2 Hardware and Software Requirements 3
4.3 Working Principle 4
4.4 Software Implementation 6
5. Results and Discussion **10**
6. Mapping POs and SDGs **15**
6.1 Program Outcomes (POs) Mapping 15
6.2 Sustainable Development Goals (SDGs) Mapping 16
7. Conclusion **16**
Bibliography **17**

Abstract

The Student Result Management System is a menu-driven application developed in Python for storing student marks and preparing results without manual calculation. In many departments, marks are still written in registers or typed into spreadsheets, and the totals, percentages and grades are worked out by hand. This takes time and a single slip can change the grade or even the pass or fail decision of a student. The present project replaces that routine with a small program that keeps every record in an SQLite database, checks the marks at the time of entry and produces the result card of any student on demand.

The program is organised in three parts: a console menu for interaction, a set of functions that handle validation, total, percentage, grade and pass or fail decisions, and a data access part that talks to the database through Python's built-in `sqlite3` module. Faculty can add a student, view a result card, correct marks, delete a record, display a rank list, see class statistics such as pass percentage and class average, and export all results to a CSV file. A student passes only when the marks in every subject are at least 40, and the grade is then decided from the overall percentage. Only modules from the Python standard library are used, so the system runs on any computer that has Python 3 installed.

Keywords: Python, SQLite, Result Management, Database, Grading, Rank List, Data Validation, Exception Handling, Console Application.

1. Introduction

Preparing results is one of the most routine and most important tasks in any educational institution. After every examination, the marks of each student in each subject have to be collected, added up, converted into a percentage and a grade, checked against the passing rules and finally published. When this is done on paper or in loosely maintained spreadsheets, the process is slow and depends heavily on the care of the person doing it. One wrong addition or one overwritten cell is enough to change the outcome for a student.

A Student Result Management System addresses this problem by keeping all academic records in one place and letting a program do the calculations. The marks are entered once. After that the system stores them, applies exactly the same rules to every student and shows the result whenever it is needed. It also makes it easy to answer questions that are tedious to answer by hand, such as who has scored the highest total, how many students have failed and what the class average is.

Python was chosen to build this system because its syntax is simple to read, it comes with a large standard library and it supports database access without any extra installation. The `sqlite3` module that ships with Python gives a complete relational database in a single file, so no database server has to be set up. Developing the project also gave practice in the topics covered in the course: functions, dictionaries and lists, string formatting, exception handling, file handling and database connectivity.

This report describes the design and implementation of the system. It handles five subjects of 100 marks each, uses a pass mark of 40 in every subject and awards grades on a seven-point scale. The application is a single-user console program meant for use by faculty or the examination section of a department.

2. Literature Review

Result processing has traditionally been handled in three ways. The first is the manual register, where marks are written by hand and totals are calculated with a calculator. The second is the spreadsheet, which removes the arithmetic but not the other weaknesses: nothing stops a user from typing a mark of 150, entering the same roll number twice or accidentally overwriting a formula. The third is a full institutional ERP or examination portal. These systems are complete, but they need a server, trained administrators and a budget, which puts them out of reach for a small department, a coaching centre or a student who only wants to understand how such a system works.

Many academic projects on result management have been built using PHP with MySQL, Java with JDBC, or Python web frameworks such as Django and Flask. These give a browser-based interface and multi-user access, but the amount of setup involved can hide the core logic from a beginner. At the other extreme, simple programs written with Python lists or text files lose their data when the program closes, or become difficult to search once the number of students grows. Standard database textbooks (Elmasri and Navathe, 2016; Silberschatz, Korth and Sudarshan, 2019) point

out that keeping data in a relational database with keys and constraints is the dependable way to guarantee consistency, and this is the approach followed here.

SQLite is a good fit for a project of this size. It is an embedded database engine that stores everything in a single file, needs no configuration and is included with Python. It supports primary keys, foreign keys, CHECK constraints and transactions, which are the features needed to keep result data correct. Its foreign key support has to be switched on for each connection, a detail that this project takes care of.

This project therefore sits between the two extremes described above. It keeps the data in a real database, validates every input, applies the pass and grade rules in one place and offers the analysis that faculty usually need (rank list, pass percentage, class average and CSV export), while remaining small enough to be read and understood completely.

3. Objectives

To design and develop a Python-based system for storing and managing student academic records.

The first objective is to build a working application in Python that stores student details and subject marks permanently in an SQLite database. The system should support the basic record operations of adding a student, viewing the record, updating marks and deleting a student, and it should prevent duplicate roll numbers and incomplete records.

To automate the calculation of total, percentage, grade and result status with proper validation.

The second objective is to remove manual calculation. Once the marks of a student are entered, the system should compute the total and percentage, decide pass or fail using the rule that every subject must be cleared, and assign a grade from a fixed scale. Marks that are outside the range 0 to 100, non-numeric entries and invalid menu choices should be handled with clear messages instead of program crashes.

To provide analysis features such as rank list, class statistics and data export.

The third objective is to give the faculty the summary information needed after results are prepared. This includes a rank list ordered by total marks, the number of students passed and failed, the pass percentage, the class average and the name of the topper, along with an option to export all results to a CSV file that can be opened in a spreadsheet.

4. Methodology

4.1 Architectural Diagram

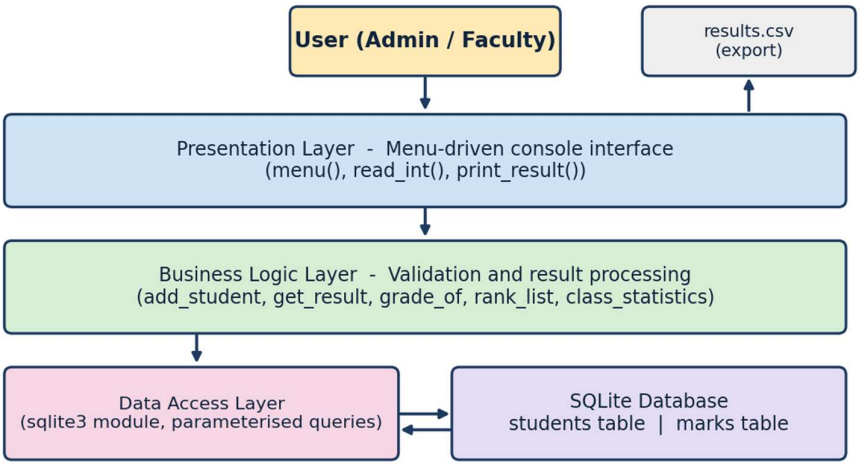


Figure 1: Architectural Diagram

The system follows a simple layered design in which each layer talks only to the layer next to it. The user works with the **Presentation Layer**, which is a numbered console menu. This layer reads the input, converts it to the right type (for example, `read_int()` keeps asking until a whole number is typed) and prints the result card, rank list or messages. It contains no calculation logic.

The **Business Logic Layer** holds the rules of the system. Functions such as `add_student()`, `get_result()`, `grade_of()`, `rank_list()` and `class_statistics()` check that marks lie between 0 and 100, compute totals and percentages, decide pass or fail and assign grades. Keeping these rules in separate functions means that they can be tested without going through the menu and reused later if a graphical or web interface is added.

The **Data Access Layer** is the only part that talks to the database. It uses the `sqlite3` module and parameterised queries (placeholders marked with `?`), so the values typed by the user are never pasted into the SQL text. The **SQLite database** file `results.db` contains the `students` and `marks` tables. The system can also write all results to `results.csv` for use outside the program.

4.2 Hardware and Software Requirements

Hardware:

- Intel/AMD processor
- Minimum 4 GB RAM
- About 100 MB of free disk space
- Standard display and input devices (monitor, keyboard)

Software:

- Python 3.8 or later (the project was developed and tested with Python 3.12)
- Python standard library modules `sqlite3` and `csv` (no external packages required)
- Any code editor or IDE (IDLE, VS Code or PyCharm)
- Operating system: Windows, Linux or macOS

4.3 Working Principle

When the program starts, `init_db()` creates the database file `results.db` with two tables if they do not already exist, and the main menu is displayed. The menu is shown repeatedly until the user selects Exit. Each menu option calls the matching function, and any error raised during the operation is caught and shown as a short message so that the program keeps running.

Database design

The data is stored in two tables, shown in Figure 2. The `students` table holds one row per student with the roll number as the primary key, so two students can never share a roll number. The `marks` table holds one row for each student and subject. Its primary key is the pair (`roll_no`, `subject`), which allows only one mark per subject for a student. A CHECK constraint restricts `score` to the range 0 to 100, and a foreign key links `roll_no` to the `students` table with `ON DELETE CASCADE`, so deleting a student also removes the marks of that student. SQLite does not enforce foreign keys unless asked, therefore `PRAGMA foreign_keys = ON` is executed every time a connection is opened.

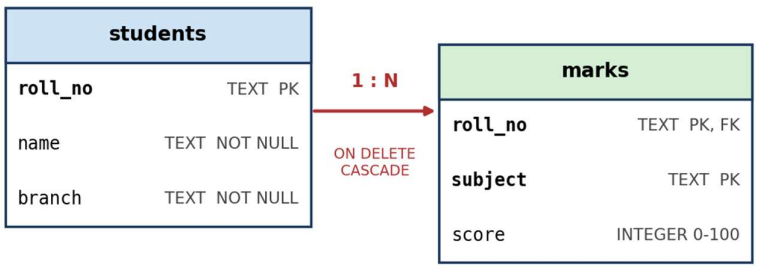


Figure 2: Database Schema

Only the raw marks are stored. The total, percentage, grade and status are calculated from the marks each time a result is requested. This means that when a mark is corrected, the result is always up to date and there is no second copy of the data that could become inconsistent.

Result computation

For a student with marks in the five subjects (Mathematics, Physics, Chemistry, English and Computer Science), the system calculates:

$$\text{Total} = \text{sum of the marks in all five subjects (out of 500)}$$

$$\text{Percentage} = (\text{Total} / 500) \times 100, \text{ rounded to two decimal places}$$

A student is declared **PASS** only if the marks in every subject are at least 40. A student who scores below 40 in even one subject is declared **FAIL** and receives grade F, whatever the overall percentage may be. For passed students the grade is taken from the percentage according to Table 1.

Table 1: Grade scale used by the system

Percentage	Grade	Remark
90 and above	O	Outstanding
80 to below 90	A+	Excellent
70 to below 80	A	Very Good
60 to below 70	B	Good
50 to below 60	C	Average
40 to below 50	D	Pass
Below 40, or below 40 in any subject	F	Fail

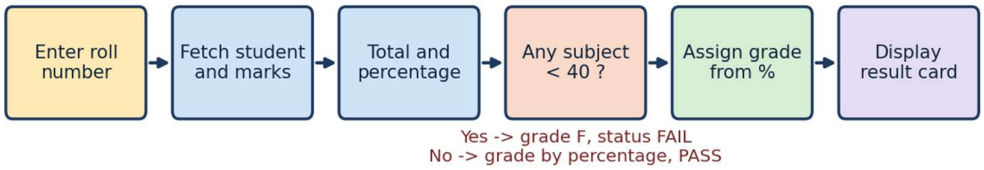


Figure 3: Steps in generating a result

Menu operations

Table 2: Menu options and the functions behind them

Option	Operation	Main function(s) used
1	Add a student with marks in all subjects	`add_student()`
2	View the result card of a student	`get_result()`, `print_result()`
3	Update the marks of one subject	`update_marks()`
4	Delete a student and the related marks	`delete_student()`
5	Display the rank list	`rank_list()`, `print_rank_list()`
6	Show class statistics	`class_statistics()`
7	Export all results to a CSV file	`export_csv()`
0	Exit the program	`menu()`

Example problem

To check the logic, five students were entered with the marks given in Table 3. The data was chosen to cover different grades and also one student who has a low mark in a single subject.

Table 3: Sample input data (marks out of 100)

Roll No	Name	Maths	Physics	Chemistry	English	CS
21CS001	Anita Rao	88	92	85	78	95
21CS002	Rahul Verma	72	65	58	70	81
21CS003	Sneha Reddy	35	48	52	60	44
21CS004	Kiran Kumar	91	89	94	86	97
21CS005	Priya Nair	66	71	59	80	74

Working these out by hand gives the values in Table 4. Sneha Reddy has 239 marks out of 500, which is 47.80%, but the 35 in Mathematics is below the pass mark, so she is declared FAIL with grade F.

Table 4: Calculated results for the sample data

Roll No	Name	Total / 500	Percentage	Grade	Status
21CS001	Anita Rao	438	87.60	A+	PASS
21CS002	Rahul Verma	346	69.20	B	PASS
21CS003	Sneha Reddy	239	47.80	F	FAIL
21CS004	Kiran Kumar	457	91.40	O	PASS
21CS005	Priya Nair	350	70.00	A	PASS

For the class as a whole, 4 of the 5 students pass (pass percentage 80%), the average percentage is (87.60 + 69.20 + 47.80 + 91.40 + 70.00) / 5 = 73.20, and the topper is Kiran Kumar with 457 marks. The program output in Section 5 matches these values.

4.4 Software Implementation

The whole system is written in a single file, `srms.py`, of about 200 lines. It imports only `sqlite3` and `csv`. The constants `SUBJECTS` and `PASS_MARK` at the top control the subject list and the passing mark, so they can be changed without touching the rest of the code. Table 5 lists the functions and what each of them does, and the complete listing follows.

Table 5: Functions in srms.py

Function	Purpose
`connect()`	Opens the database and enables foreign key checking
`init_db()`	Creates the `students` and `marks` tables if they are missing
`grade_of(pct)`	Returns the grade for a given percentage

Function	Purpose
`add_student()`	Validates the marks and inserts the student and the five marks
`get_result()`	Reads a student's marks and calculates total, percentage, grade and status
`update_marks()`	Changes the mark of one subject for a student
`delete_student()`	Removes a student; the marks are removed by the cascade rule
`all_results()`, `rank_list()`	Collect all results and sort them (passed students first, then by total)
`class_statistics()`	Counts passed and failed students and finds the average and topper
`export_csv()`	Writes the rank list to a CSV file
`print_result()`, `print_rank_list()`	Format the result card and the rank list for the screen
`read_int()`	Keeps asking until the user types a whole number
`menu()`	Main loop that shows the menu and calls the other functions

Program: srms.py

```
"""Student Result Management System (console based, SQLite backed)."""
import sqlite3
import csv

DB_NAME = "results.db"
SUBJECTS = ["Mathematics", "Physics", "Chemistry", "English", "Computer Science"]
PASS_MARK = 40

def connect():
    conn = sqlite3.connect(DB_NAME)
    conn.execute("PRAGMA foreign_keys = ON")
    return conn

def init_db():
    with connect() as conn:
        conn.execute("""CREATE TABLE IF NOT EXISTS students (
                            roll_no TEXT PRIMARY KEY,
                            name    TEXT NOT NULL,
                            branch  TEXT NOT NULL)""")
        conn.execute("""CREATE TABLE IF NOT EXISTS marks (
                            roll_no TEXT,
                            subject TEXT,
                            score   INTEGER CHECK (score BETWEEN 0 AND 100),
                            PRIMARY KEY (roll_no, subject),
                            FOREIGN KEY (roll_no) REFERENCES students(roll_no)
                                ON DELETE CASCADE)""")

def grade_of(pct):
    if pct >= 90: return "O"
    if pct >= 80: return "A+"
    if pct >= 70: return "A"
    if pct >= 60: return "B"
    if pct >= 50: return "C"
    if pct >= PASS_MARK: return "D"
    return "F"

def add_student(roll, name, branch, scores):
    if len(scores) != len(SUBJECTS):
        raise ValueError("Marks required for all %d subjects" % len(SUBJECTS))
    for s in scores:
        if not 0 <= s <= 100:
            raise ValueError("Marks must be between 0 and 100")
```

```
    with connect() as conn:
        conn.execute("INSERT INTO students VALUES (?,?,?)", (roll, name, branch))
        conn.executemany("INSERT INTO marks VALUES (?,?,?)",
                         [(roll, sub, sc) for sub, sc in zip(SUBJECTS, scores)])

def get_result(roll):
    with connect() as conn:
        st = conn.execute("SELECT name, branch FROM students WHERE roll_no=?",
                          (roll,)).fetchone()
        if st is None:
            return None
        rows = conn.execute("SELECT subject, score FROM marks WHERE roll_no=?",
                            (roll,)).fetchall()
    marks = dict(rows)
    total = sum(marks.values())
    pct = total / (100 * len(SUBJECTS)) * 100
    passed = all(v >= PASS_MARK for v in marks.values())
    return {"roll": roll, "name": st[0], "branch": st[1], "marks": marks,
            "total": total, "pct": round(pct, 2),
            "grade": grade_of(pct) if passed else "F",
            "status": "PASS" if passed else "FAIL"}

def update_marks(roll, subject, score):
    if not 0 <= score <= 100:
        raise ValueError("Marks must be between 0 and 100")
    with connect() as conn:
        cur = conn.execute(
            "UPDATE marks SET score=? WHERE roll_no=? AND subject=?",
            (score, roll, subject))
        return cur.rowcount > 0

def delete_student(roll):
    with connect() as conn:
        return conn.execute("DELETE FROM students WHERE roll_no=?",
                            (roll,)).rowcount > 0

def all_results():
    with connect() as conn:
        rolls = [r[0] for r in conn.execute("SELECT roll_no FROM students")]
    return [get_result(r) for r in rolls]

def rank_list():
    res = all_results()
    res.sort(key=lambda r: (r["status"] == "FAIL", -r["total"]))
    return res

def class_statistics():
    res = all_results()
    if not res:
        return None
    passed = sum(1 for r in res if r["status"] == "PASS")
    avg = sum(r["pct"] for r in res) / len(res)
    topper = max(res, key=lambda r: r["total"])
    return {"students": len(res), "passed": passed, "failed": len(res) - passed,
            "pass_pct": round(passed / len(res) * 100, 2),
            "average": round(avg, 2), "topper": topper["name"]}

def export_csv(path="results.csv"):
    with open(path, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["Roll", "Name", "Branch"] + SUBJECTS +
                   ["Total", "Percentage", "Grade", "Status"])
        for r in rank_list():
            w.writerow([r["roll"], r["name"], r["branch"]] +
                       [r["marks"][s] for s in SUBJECTS] +
                       [r["total"], r["pct"], r["grade"], r["status"]])
    return path

def print_result(r):
    print("\n" + "=" * 46)
    print("        STUDENT RESULT CARD")
    print("=" * 46)
    print("Roll No : %s" % r["roll"])
    print("Name    : %s" % r["name"])
    print("Branch  : %s" % r["branch"])
    print("-" * 46)
    print("%-22s %8s %10s" % ("Subject", "Marks", "Result"))
    for sub in SUBJECTS:
        m = r["marks"][sub]
        print("%-22s %8d %10s" % (sub, m, "Pass" if m >= PASS_MARK else "Fail"))
    print("-" * 46)
    out_of = 100 * len(SUBJECTS)
```

```
    print("Total: %d/%d   Percentage: %.2f%%" % (r["total"], out_of, r["pct"]))
    print("Grade: %s   Status: %s" % (r["grade"], r["status"]))
    print("=" * 46)

def print_rank_list():
    fmt = "%-4s %-10s %-18s %6s %7s %5s"
    print("\n" + fmt % ("Rank", "Roll", "Name", "Total", "%", "Grade"))
    for i, r in enumerate(rank_list(), 1):
        print(fmt % (i, r["roll"], r["name"], r["total"],
                     "%.2f" % r["pct"], r["grade"]))

def read_int(prompt):
    while True:
        try:
            return int(input(prompt))
        except ValueError:
            print("Please enter a whole number.")

def menu():
    init_db()
    while True:
        print("\n--- STUDENT RESULT MANAGEMENT SYSTEM ---")
        print("1. Add student   2. View result   3. Update marks")
        print("4. Delete student 5. Rank list    6. Class statistics")
        print("7. Export to CSV  0. Exit")
        ch = input("Enter choice: ").strip()
        try:
            if ch == "1":
                roll = input("Roll no: ").strip()
                name = input("Name: ").strip()
                branch = input("Branch: ").strip()
                scores = [read_int("  %s marks: " % s) for s in SUBJECTS]
                add_student(roll, name, branch, scores)
                print("Student added successfully.")
            elif ch == "2":
                r = get_result(input("Roll no: ").strip())
                print_result(r) if r else print("Student not found.")
            elif ch == "3":
                roll = input("Roll no: ").strip()
                for i, s in enumerate(SUBJECTS, 1):
                    print(i, s)
                num = read_int("Subject number: ")
                if not 1 <= num <= len(SUBJECTS):
                    raise ValueError("Subject number must be 1 to %d"
                                     % len(SUBJECTS))
                sub = SUBJECTS[num - 1]
                ok = update_marks(roll, sub, read_int("New marks: "))
                print("Marks updated." if ok else "Record not found.")
            elif ch == "4":
                ok = delete_student(input("Roll no: ").strip())
                print("Deleted." if ok else "Student not found.")
            elif ch == "5":
                print_rank_list()
            elif ch == "6":
                s = class_statistics()
                print(s if s else "No records yet.")
            elif ch == "7":
                print("Exported to", export_csv())
            elif ch == "0":
                print("Goodbye!")
                break
            else:
                print("Invalid choice.")
        except sqlite3.IntegrityError:
            print("Error: that roll number already exists.")
        except (ValueError, IndexError) as e:
            print("Error:", e)

if __name__ == "__main__":
    menu()
```

5. Results and Discussion

The program was run with the sample data of Table 3 and each feature was tested through the menu. The figures below are the actual outputs of the program. Figure 4 shows a student being added; the prompts ask for the roll number, name, branch and the marks in each of the five subjects.

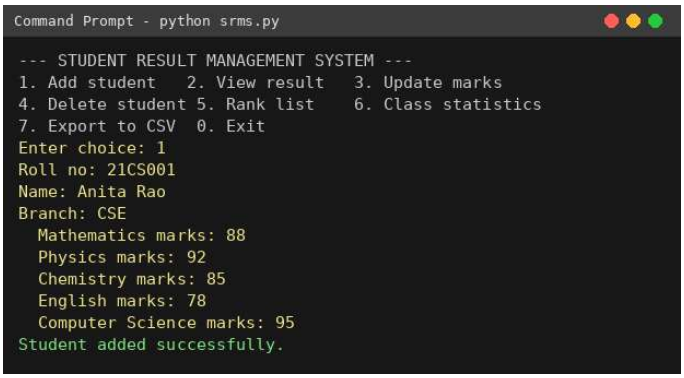


Figure 4: Output (Adding a student record)

Figure 5 shows the result card of Anita Rao. The total of 438 out of 500 gives 87.60%, which falls in the A+ band, and every subject is above 40, so the status is PASS.

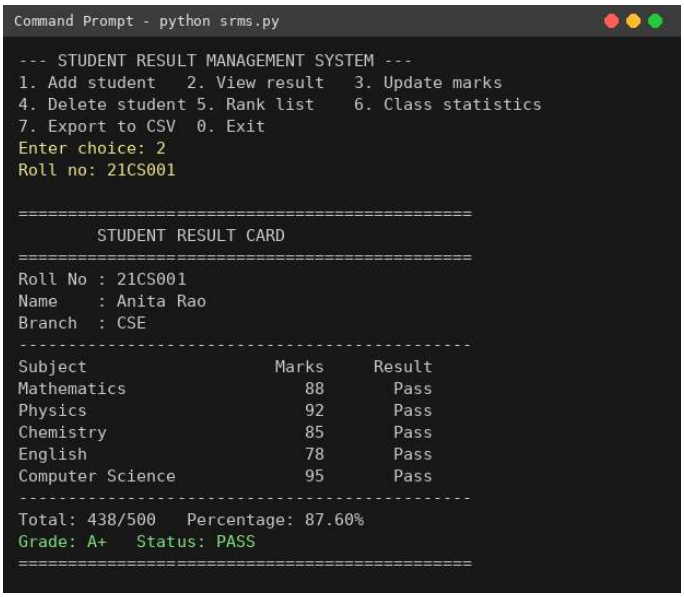


Figure 5: Output (Result card of a passed student)

Figure 6 shows the result card of Sneha Reddy. Her percentage is 47.80%, yet the result is FAIL with grade F because the mark in Mathematics is 35, below the pass mark of 40. This confirms that the subject-wise passing rule is applied.

```
Command Prompt - python srms.py

--- STUDENT RESULT MANAGEMENT SYSTEM ---
1. Add student   2. View result   3. Update marks
4. Delete student 5. Rank list    6. Class statistics
7. Export to CSV  0. Exit
Enter choice: 2
Roll no: 21CS003

============================================
        STUDENT RESULT CARD
============================================
Roll No : 21CS003
Name    : Sneha Reddy
Branch  : CSE
--------------------------------------------
Subject                  Marks     Result
Mathematics                 35       Fail
Physics                     48       Pass
Chemistry                   52       Pass
English                     60       Pass
Computer Science            44       Pass
--------------------------------------------
Total: 239/500   Percentage: 47.80%
Grade: F   Status: FAIL
============================================
```

Figure 6: Output (Result card of a student failing in one subject)

Figure 7 shows the rank list and the class statistics. Students are ordered by total marks, with students who have failed placed after those who have passed. The statistics agree with the values worked out by hand in Section 4.3.

```
Command Prompt - python srms.py

--- STUDENT RESULT MANAGEMENT SYSTEM ---
1. Add student   2. View result   3. Update marks
4. Delete student 5. Rank list    6. Class statistics
7. Export to CSV  0. Exit
Enter choice: 5

Rank Roll       Name               Total       % Grade
1    21CS004    Kiran Kumar          457   91.40     O
2    21CS001    Anita Rao            438   87.60    A+
3    21CS005    Priya Nair           350   70.00     A
4    21CS002    Rahul Verma          346   69.20     B
5    21CS003    Sneha Reddy          239   47.80     F

Enter choice: 6
{'students': 5, 'passed': 4, 'failed': 1, 'pass_pct': 80.0, 'average': 73.2, 'topper': 'Kiran Kumar'}
```

Figure 7: Output (Rank list and class statistics)

In Figure 8 the Mathematics mark of Sneha Reddy is corrected from 35 to 55. When the result is viewed again, the total changes to 259, the percentage to 51.80%, and the status to PASS with grade C. The new result is produced straight away because it is calculated from the stored marks each time.

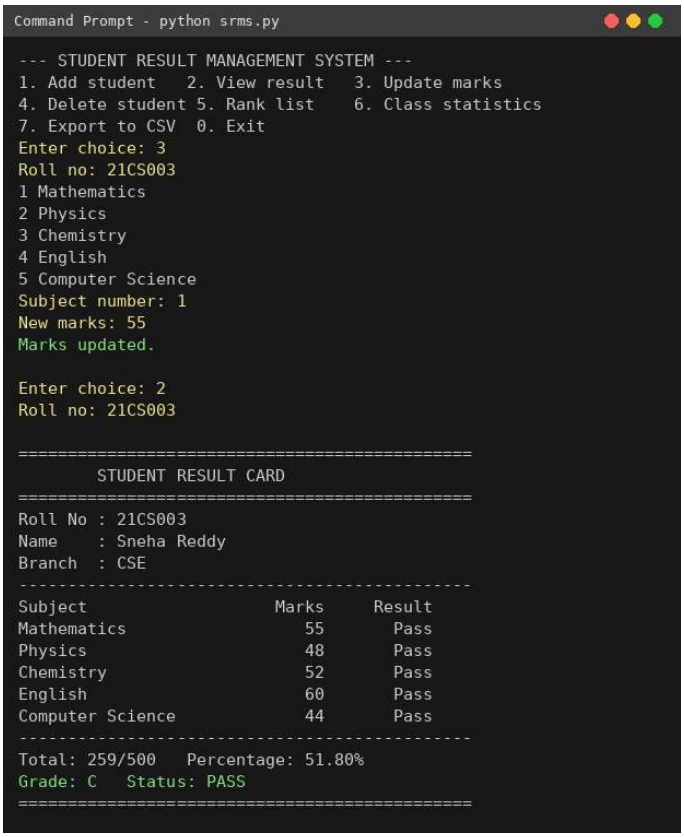


Figure 8: Output (Updating marks and viewing the new result)

Figure 9 shows how the system reacts to wrong input. A duplicate roll number is refused, a non-numeric mark makes the program ask again, a mark of 150 is rejected, an invalid subject number is reported and a search for a roll number that does not exist returns a message. In each case the program continues to run.

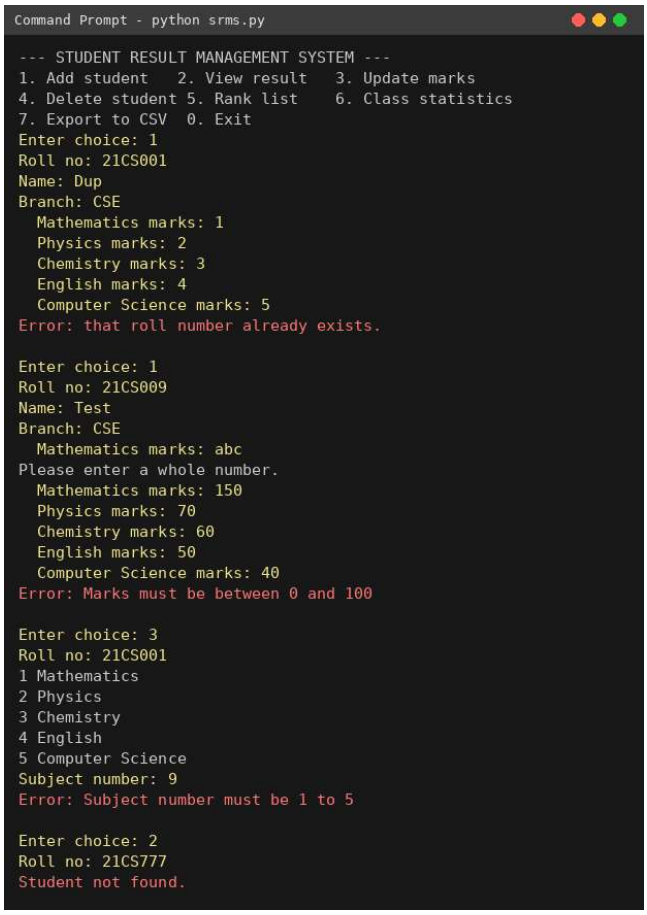


Figure 9: Output (Input validation and error handling)

Figure 10 shows a student being deleted. A second attempt to delete the same roll number reports that the student is not found, and the rank list no longer contains the deleted student. A check of the database confirmed that the marks of that student were also removed by the cascade rule.

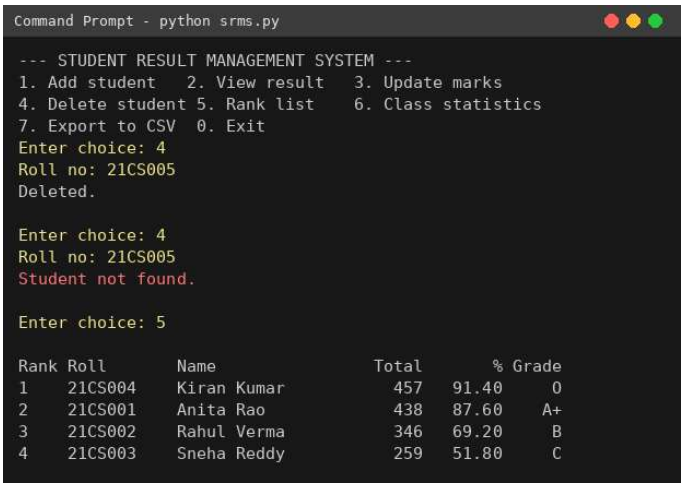


Figure 10: Output (Deleting a student record)

Test cases

Table 6: Test cases and observed results

No.	Test	Expected behaviour	Observed	Result
1	Add a valid student	Record saved	"Student added successfully."	Pass
2	Add a duplicate roll number	Rejected	"that roll number already exists"	Pass
3	Marks above 100 (150)	Rejected	"Marks must be between 0 and 100"	Pass
4	Non-numeric marks (abc)	Asks again	"Please enter a whole number."	Pass
5	View result of 21CS001	438/500, 87.60%, A+	Same as expected	Pass
6	One subject below 40 (21CS003)	FAIL, grade F	FAIL, grade F at 47.80%	Pass
7	Unknown roll number	Message shown	"Student not found."	Pass
8	Update Maths of 21CS003 to 55	Result recalculated	259, 51.80%, C, PASS	Pass
9	Invalid subject number (9)	Rejected	"Subject number must be 1 to 5"	Pass
10	Delete 21CS005	Student and marks removed	Both removed	Pass
11	Rank list	Descending by total	Sorted correctly	Pass
12	Export to CSV	File with header and one row per student	`results.csv` created	Pass

Summary of Results

All twelve test cases behaved as expected. The totals, percentages, grades and pass or fail status produced by the program matched the hand calculation for all five sample students, and the class statistics (5 students, 4 passed, 1 failed, pass percentage 80%, average 73.20%, topper Kiran Kumar) were correct. Invalid inputs were handled with messages and never stopped the program.

Interpretation

The case of Sneha Reddy shows why the pass rule is applied to each subject and not only to the percentage: a student with 47.80% still fails when one subject is below 40. After the correction in Mathematics the same student passes with grade C, which shows that results are derived from the stored marks and not saved separately. The constraints in the database (primary keys, the 0 to 100 check and the foreign key) act as a second line of defence behind the validation in the Python code, so wrong data does not enter the tables even if one check is missed.

Achievement of Objectives

- **Objective 1:** Student records are stored permanently in SQLite and can be added, viewed, updated and deleted, with duplicate roll numbers prevented.
- **Objective 2:** Total, percentage, grade and status are calculated automatically, and invalid marks and menu choices are handled without errors.
- **Objective 3:** The rank list, class statistics and CSV export are available from the menu and give correct results.

Implications

The system can be used in a small department, a laboratory or a coaching class that needs a simple and dependable way to prepare results. Because the logic is separated from the menu, the same functions can later be used behind a graphical or web interface. The project also works as a teaching example of how Python and a relational database can be combined in a complete application.

6. Mapping POs and SDGs

6.1 Program Outcomes (POs) Mapping

Table 7: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied programming, database and SQL concepts to build a working result processing system in Python.
PO2	Problem Analysis	Analysed the manual result preparation process and turned the pass mark, grade scale and ranking rules into precise program logic.
PO3	Design/Development of Solutions	Designed a layered structure and a two-table database, and implemented add, view, update, delete, rank, statistics and export operations.
PO4	Conduct Investigations of Complex Problems	Tested the system with valid, invalid and boundary inputs, such as a student who fails one subject but scores above 40% overall, and verified the outputs.
PO5	Engineering Tool Usage	Used Python 3, the `sqlite3` and `csv` modules and a code editor for development, debugging and testing.
PO8	Individual and Collaborative Team Work	Shared the work of coding, testing and documentation and combined the parts through regular review.
PO9	Communication	Presented the design with diagrams, tables and program outputs, and documented the work in this report.
PO11	Life-Long Learning	Learnt SQLite constraints, exception handling and CSV handling from documentation and practice beyond the classroom syllabus.

6.2 Sustainable Development Goals (SDGs) Mapping

Table 8: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG 4	Quality Education	Gives institutions accurate academic records and quick results, and serves as a hands-on example for learning programming and databases.
SDG 8	Decent Work and Economic Growth	Builds software development and problem-solving skills that improve employability in the IT sector.
SDG 9	Industry, Innovation and Infrastructure	Supports the move of record keeping in educational institutions from paper to digital systems.
SDG 12	Responsible Consumption and Production	Reduces the use of paper registers and printed mark lists through digital storage and CSV export.

7. Conclusion

This project developed a Student Result Management System in Python with SQLite as the database. The system stores student marks, validates them at entry, calculates the total, percentage, grade and pass or fail status using fixed rules, and provides a rank list, class statistics and CSV export. Testing with sample data showed that the results agree with manual calculation and that wrong inputs are handled without stopping the program. Keeping the marks as the only stored data and deriving everything else from them kept the design simple and the results consistent.

The present version has some limitations. It is a single-user console program, it does not ask for a login, and the list of five subjects is fixed in the code. It does not handle semester-wise records or credit-based grade points. These can be taken up as future work, along with a graphical interface using Tkinter or a web interface using Flask, user login with separate roles for faculty and students, subject lists that can be configured, CGPA and SGPA calculation, printable PDF result cards, and automatic backup of the database. With these additions the system could be used for the complete result processing of a department.

Bibliography

Online Resources

1. Python Software Foundation – Python 3 Documentation. https://docs.python.org/3/
2. Python Software Foundation – sqlite3: DB-API 2.0 interface for SQLite databases. https://docs.python.org/3/library/sqlite3.html
3. Python Software Foundation – csv: CSV File Reading and Writing. https://docs.python.org/3/library/csv.html
4. SQLite Documentation – SQLite Foreign Key Support. https://www.sqlite.org/foreignkeys.html
5. W3Schools – Python Tutorial. https://www.w3schools.com/python/

Books and Reference Texts

1. Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.
2. Matthes, E. (2023). Python Crash Course (3rd ed.). No Starch Press.
3. Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education.
4. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.
5. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.